

# Edge cases (a task that needs no tool at all) — Solution

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

**Story** — `no_tool_needed_practice.py`: an agent that always reaches for a tool, even when general knowledge alone answers the question, wastes cost and time on every single call. Confirming zero tool calls for an obviously-answerable prompt is what proves the loop isn't reflexively tool-happy. **If not:** the Build Task's "task needs no tool → direct answer, zero tool calls" Test Case would be the first time you ever checked this, with no smaller version to trust it against.

All examples below assume `run_agent()` from `build_react_loop` (returns an `AgentResult` with `final_answer` and `steps`), with `get_weather` and `celsius_to_fahrenheit` as the only 2 registered tools.

## Basic Version

---

```
# no_tool_needed_practice.py
result = run_agent("What's the capital of France?", max_iterations=5)
print(result.final_answer)
print(result.steps)
```

### Expected output:

```
The capital of France is Paris.
[]
```

`result.steps` comes back empty because the model's very first response already had no `tool_calls` — it answered directly. This works correctly and confirms the exercise's core claim, but it's something you read once, not something a test suite can check for you automatically.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

### Approach 1 — one real assertion

```
# no_tool_needed_practice.py
def test_no_tool_call_for_general_knowledge():
    result = run_agent("What's the capital of France?", max_iterations=5)
    assert len(result.steps) == 0
    assert "Paris" in result.final_answer
```

Expected output when run with pytest:

1 passed

### Approach 2 — a small list of clearly-no-tool prompts

```
# no_tool_needed_practice.py
NO_TOOL_PROMPTS = [
    "What's the capital of France?",
    "What year did World War II end?",
    "Who wrote Romeo and Juliet?",
]

def test_clear_prompts_never_call_a_tool():
    # why: several prompts in one test catches a case where the model
    # behaves correctly on one question but not another.
    for prompt in NO_TOOL_PROMPTS:
        result = run_agent(prompt, max_iterations=5)
        assert len(result.steps) == 0, f"unexpected tool call for: {prompt}"
```

Expected output when run with pytest:

1 passed

**Difference from Basic:** Basic reads printed output once and trusts it looks right. Approach 1 turns that same check into a real `assert`, so it can run in CI and fail loudly if it ever stops being true.

Approach 2 widens the check to several prompts in one test, catching a case where the model behaves correctly on one question but not another — a single example was never proof of the general claim, just one data point toward it.

**Which one should you actually write?** Approach 2's kind of test — a short list of clearly-no-tool prompts, each asserted to produce zero tool calls — is exactly what belongs in `test_agent.py` for the Build Task, matching the Test Cases table's "Task needs no tool → Direct answer, zero tool calls" row. Worth knowing beyond this exercise: a genuinely borderline prompt (like "is Paris generally a warm city?") can go either way depending on how precisely your Doc06 tool descriptions are worded — if you ever see a clearly-no-tool prompt start calling a tool it shouldn't, go check the tool's *description* first, before assuming your loop itself is broken.